

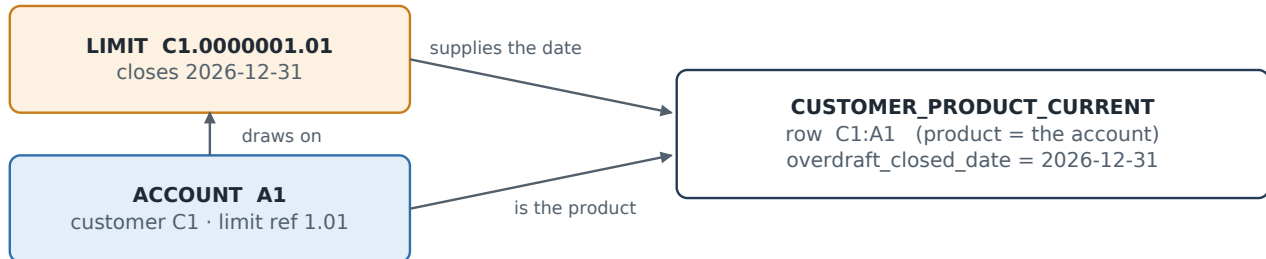
Overdraft Closed Date

Solution design · consumer-lo CDP · branch OVER_DRAFT_CLOSED_DATE

30 September 2026 · Status: **designed, built and unit-tested; not yet deployable** (two business placeholders to fill, Spark run pending)

1. The problem and the decision

The first version keyed the overdraft on the **limit**: its customer-product row was <customer>:<limitId>. That is the wrong product. An overdraft is an **account** that borrows against a limit: the account is the product, and the limit only supplies the closed date.



Decision. The product id is the **account id**. The row is <customerId>:<accountId>, the same row the account delete cascade already closes. Each account row records **which limit it draws on** (d:oref), and a limit finds its accounts through the **customer inside the limit id** plus that label.

2. How a limit finds its accounts

A T24 limit id carries both halves of the match. Its first token is the customer; the rest is the limit reference that an account records in its own LIMIT.REF field.

C1 . 0000001.01 $\xrightarrow{\text{drop leading zeros}}$ C1 + 1.1

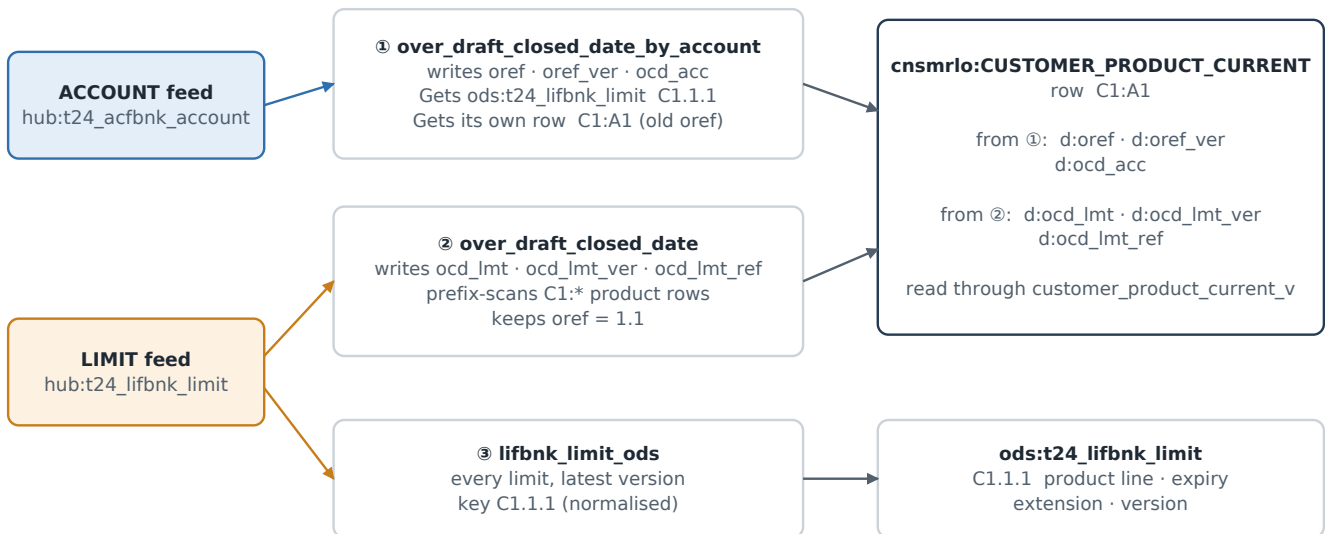
customer limit reference matched against the account's own ref '1.01' → 1.1
(T24 pads inside a limit id; an account's LIMIT.REF may not be padded)

The account job copies each account's normalised reference onto its product row as d:oref. On a limit change, the limit job reads every product row of that customer (one key-range scan) and keeps the rows whose oref is the limit's reference. **The product row is its own index**; there is no separate customer → accounts table.

Product row	d:oref	Result for limit C1.0000001.01
C1:A1	1.1	affected
C1:A2	2.1	ignored: another limit
C1:MM77	—	ignored: not an account on a limit
C2:A3	1.1	never read: another customer

3. The three jobs

Two feeds. Every job has its own checkpoint, so none waits for another.



#	Job	Runs when	Does
①	over_draft_closed_date_by_account	an account changes	Writes the account's current limit (oref) and that limit's closed date (ocd_acc), looked up in the limit ODS. Empties both when the account leaves its limit. Skips accounts that are not, and never were, on a limit.
②	over_draft_closed_date	a limit changes	Finds the customer's accounts whose oref is this limit's reference and writes the new date (ocd_lmt), labelled with the reference it is for (ocd_lmt_ref).
③	lifbnk_limit_ods	a limit changes	Keeps the latest raw state of every limit (product line, expiry, extension, version), keyed <customer>.<normalised ref> so ① can build the key from LIMIT.REF. Deployable on its own.

Why two writers?

Limit's date changes	→ only the LIMIT record changes	→ job 2 catches it
Account attached, moved or left	→ only the ACCOUNT record changes	→ job 1 catches it

Why the limit ODS keys on the normalised reference

T24 zero-pads the reference inside a limit id (0000001.01) but an account's LIMIT.REF need not be (1.01), so the account job could not rebuild the raw id. Both sides strip the leading zeros of each part the same way, and every place that builds the key is marked KEEP IN STEP.

4. Writing the date: one set of cells per job, one view

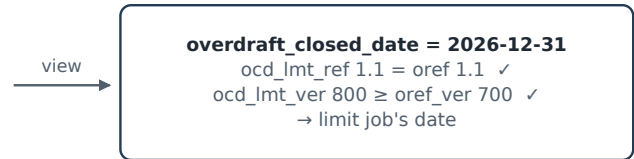
Each job writes only its own cells on the account's row, so a replay of one can never overwrite the other. Readers use one view, which decides between them.

written by ① account job

d:oref	d:oref_ver	d:ocd_acc
1.1	700	2026-06-30

written by ② limit job

d:ocd_lmt_ref	d:ocd_lmt_ver	d:ocd_lmt
1.1	800	2026-12-31



row C1:A1 in CUSTOMER_PRODUCT_CURRENT

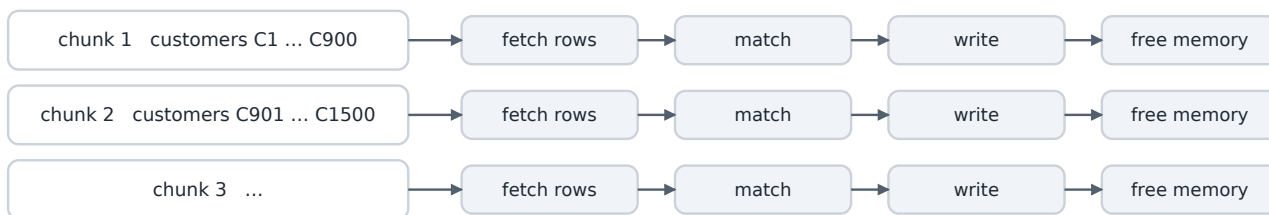
Rule	Why
The limit job's date counts only while ocd_lmt_ref = oref	A limit change processed before the account job records a move writes a date for a limit the account has left. The label shows it is stale, so the view ignores it, whichever job ran first.
...and only while ocd_lmt_ver ≥ oref_ver	An account change after the limit's last change is newer information; its own date (ocd_acc) stands.
The account job stamps its cells with the account's version	A later move, to another limit or to none, must always win over its earlier cells.
The limit job stamps with greatest(limit version, oref_ver)	Its cells are shared by every limit the account is ever on; a new limit's version can be older than the previous one's.
An empty value clears; the view turns it into NULL	The sink writes nothing for a NULL, so clearing needs an empty value.
closed_date = COALESCE(overdraft_closed_date, account_closed_date)	The account's own closed date (d:acd) is a separate fact with a separate owner, combined only in the view.

5. Walk-through

Event	Job ① writes	Job ② writes	View shows
A1 attached to limit ref 1.1 (limit unchanged)	oref 1.1, ocd_acc = L1's date	—	L1's date
Limit 1.1 extended	—	ocd_lmt = new date, ref 1.1	new date
A1 moved to ref 2.1	oref 2.1, ocd_acc = L2's date	—	L2's date (ref 1.1 ≠ 2.1)
Old limit 1.1 changes just before ① runs	—	ocd_lmt for ref 1.1	L2's date: label is stale
A1 taken off any limit	oref and ocd_acc emptied	—	no overdraft date
Limit arrives after the account	oref set, ocd_acc empty	finds A1 by oref	the limit's date

6. Scaling: large customers

Job ② reads **all** product rows of each customer in the batch. For a corporate customer that can be tens of thousands of rows, all held on the Spark driver. The batch is therefore processed in **chunks**:



- **One customer is never split across chunks**, so a limit is never separated from its accounts.
- `maxRowsPerChunk` (default 100,000) bounds each chunk. A single customer above the cap stops the batch with a message naming the customer and the setting. Chunks already written are rewritten unchanged when the batch is retried.
- HBase is not held open while Spark works: each chunk's read is closed before its join and write.
- Rule for any statement on such a join: it must not total across customers.

7. Closing

```
Account A1 closed  →  acfbnk_account_delete_cascade  →  C1:A1  status = CLOSED
                    (the overdraft IS the account, so this closes it too)

Limit L1 deleted   →  no cascade  (lifbnk_limit_delete_cascade removed)
```

The limit delete cascade closed `<customer>:<limit>` rows, which no longer represent a product, and it must not close the account: losing an overdraft does not close a current account. A catalog test asserts the account cascade is the only delete cascade.

Dormant today. Both the account hub and the limit hub have the GoldenGate delete marker (`opTypeField`) switched off. No delete reaches any job until the hub owner turns it on.

8. Engine changes (hub-sql-to-hbase)

Setting	Meaning
<code>source.join[].match: prefix</code>	Read every <code><key>:<child></code> row under each key with one multi-range scan. The row-key column carries the full row key. Unsalted tables only. Default: key (exact Get, as before).
<code>source.join[].maxRowsPerChunk</code>	Prefix joins only: run the batch in chunks of at most this many rows (default 100,000). At most one prefix join per pipeline.

9. Risks and where they stand

Risk	Status
Account attached to an unchanged limit never gets a date	Fixed: account job ①
Account leaves its limit and keeps the old date	Fixed: ① clears oref and ocd_acc
Limit change processed before a move is recorded	Fixed: ocd_lmt_ref must match oref
Two writers race on one cell / replays	Fixed: one set of cells per job, view resolves
Large customer overloads the driver	Fixed: row-capped chunks; oversized customer fails loudly
Phantom limit rows from the limit delete cascade	Fixed: cascade removed
Limit deleted while the account stays open	Open: the date stays, and the ODS row stays too, so ① rewrites it on the next account change. Fix: treat the limit tombstone as no overdraft in ② and ③.
Overdraft filter and date rule live in two statements	Open, accepted: both marked KEEP IN STEP

10. Before go-live

#	Item	Owner	Effect until done
1	Overdraft product line: replace SET_ME_OVERDRAFT_PRODUCT_LINE	Business	Jobs ① and ② stop, naming it
2	Account LIMIT.REF tag: replace SET_ME_ACCOUNT_LIMIT_REF_TAG	T24 team	Job ① stops, naming it
3	Closed-date rule: extension date, else expiry (assumed)	Business	Assumption stands
4	Source of the account's own closed date (d:acd)	Business	closed_date only ever shows the overdraft date
5	closed_date order: overdraft first (as now), or the earlier date	Business	Overdraft first
6	Start ① before ②, and ③ first of all	Engineering	② replays the whole limit hub on its first run; started before ① it finds no oref and writes nothing until each limit next changes
7	First runs read whole hubs: raise maxKeysPerBatch for ① and ②, or start in phases	Engineering	The first batch fails over the 200,000-key ceiling
8	Delete markers on the account and limit hubs	Hub owner	Account close never fires
9	Re-create the Hive table (external, safe), then create the view	Platform	Impala cannot see the new columns
10	One Spark run on JDK 17 or the dev cluster	Engineering	Jobs and chunk loop unproven end to end

11. Verification

- Full multi-module build green: 10 modules, about 680 tests.
- Catalog tests: the three jobs resolve; ① Gets the ODS row ③ writes and its own product row; ② prefix-scans the product table, chunked; each job writes only its own cells; the Hive mapping and the view cover every overdraft cell.

- Every statement and both of ①'s lookup keys parse under Spark 3.3's grammar; a broken control is rejected.
- Prefix scan and chunking run against the in-memory test HBase: chunk boundaries, customers with no rows, the 10/100 ordering, an oversized customer, an empty batch.
- **Not verified:** a Spark run (Spark 3.3 cannot start on this JDK 21), and the DDL and view on a real Hive and Impala.

Commit	Change
5c8bb20	Product keyed on the account; prefix join in the engine
985708d	One set of cells per job; Impala view; Hive mapping
5b1bd98	Row-capped chunks for prefix joins
5b6c700	Remove lifbnk_limit_delete_cascade
7415c25	Separate overdraft and account closed dates; closed_date in the view
fcbbabd	oref on the product row replaces the index; account job; limit ODS